Reviewer Pack — Asymptotic Dimension and Resolution Length for Tseitin Formu…

Entry e708abcb82c7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 06:34:51 UTC

Asymptotic Dimension and Resolution Length for Tseitin Formulas

Entry ID: `e708abcb82c7` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-08 06:34:51 UTC

1. Conjecture

Field A (mathematical branch): Geometric Group Theory (Asymptotic Dimension) **Field B** (complexity object): Resolution Complexity of Tseitin Formulas

Statement:

> For a graph G, let ν(G) be its asymptotic dimension. The Tseitin formula on G requires Resolution length ≥ 2^Ω(ν(G)).

Rationale (proposer's reasoning):

> Higher asymptotic dimension indicates more complex geometric structure, which increases the difficulty of resolving the formula, leading to exponential growth in proof length.

Taxonomy category: `TSEITIN_RES` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `6a1fa8537b245c85`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math
from itertools import combinations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(matrix):
        n = len(matrix)
        for i in range(n):
            max_row = i
            for j in range(i+1, n):
                if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                    max_row = j
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
            pivot = matrix[i][i]
            if pivot == 0:
                return None  # Singular matrix, no solution
            for j in range(i+1, n):
                factor = matrix[j][i] / pivot
                for k in range(n):
                    matrix[j][k] -= factor * matrix[i][k]
        return matrix

    def resolution_length(graph):
        n = len(graph)
        matrix = [[0] * (n + 1) for _ in range(n)]
        for i in range(n):
            for j in range(i+1, n):
                if graph[i][j]:
                    matrix[i][j] = -1
                    matrix[j][i] = -1
                    matrix[i][-1] += 1
                    matrix[j][-1] += 1

        result = gaussian_elimination(matrix)
        if result is None:
            return float('inf')

        rank = sum(1 for row in result if any(row))
        return n - rank

    def asymptotic_dimension(graph):
        n = len(graph)
        for dim in range(n + 1):
```

```
            coverings = combinations(range(n), dim)
            if all(any(graph[i][j] for i, j in combinations(cov, 2)) for cov in coverings):
                return dim
        return n

    def generate_graph(n, dim):
        graph = [[0] * n for _ in range(n)]
        vertices = list(range(n))
        random.shuffle(vertices)
        for i in range(dim):
            subset = vertices[:i+1]
            for u, v in combinations(subset, 2):
                graph[u][v] = graph[v][u] = 1
        return graph

    n_values = [5, 10, 15, 20, 30, 40]
    total_length = 0
    instances_tested = 0

    for n in n_values:
        for _ in range(5):  # Ensure at least 5 instances per seed
            graph = generate_graph(n, random.randint(1, min(n-1, 3)))
            dim = asymptotic_dimension(graph)
            length = resolution_length(graph)
            total_length += length
            instances_tested += 1

    mean_length = total_length / instances_tested
    conjecture_holds = mean_length >= 2 ** (0.5 * n_values[-1])
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "resolution_length",
        "metric_value": mean_length,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_length = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_length} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_length} std=0.0 support_fraction={support_fraction:.2
```

```
    else:
        first_failing_seed = next(seed for seed, r in enumerate(results) if not r["conjecture_hold
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Increase timeout duration and re-run test with extended computation limits

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	106138
2	propose	ollama_remote	qwen3:8b	0	0	127612
3	preregistration	ollama_remote	qwen3:8b	0	0	19939
4	novelty	ollama_remote	qwen3:8b	0	0	16724
5	novelty	ollama_remote	qwen3:8b	0	0	10694
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15204
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10811
8	judge	ollama_remote	qwen3:8b	0	0	13210

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 320332 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/e708abcb82c7.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e708abcb82c7.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/e708abcb82c7.tar.gz` (if generated)